Deployment architecture at a glance

Python + Streamlit + YOLOv8 • system boundaries, runtime modules, data movement, and deployment target

Problem. Reviewing road footage for helmet-related violations can be repetitive. Solution. A Streamlit application uses trained YOLOv8 weights to analyse supported media and present detections and summaries.

REPORTED APPLICATION MODES

SINGLE IMAGE | BATCH | VIDEO | LIVE CAMERA | ANALYTICS

Application layer

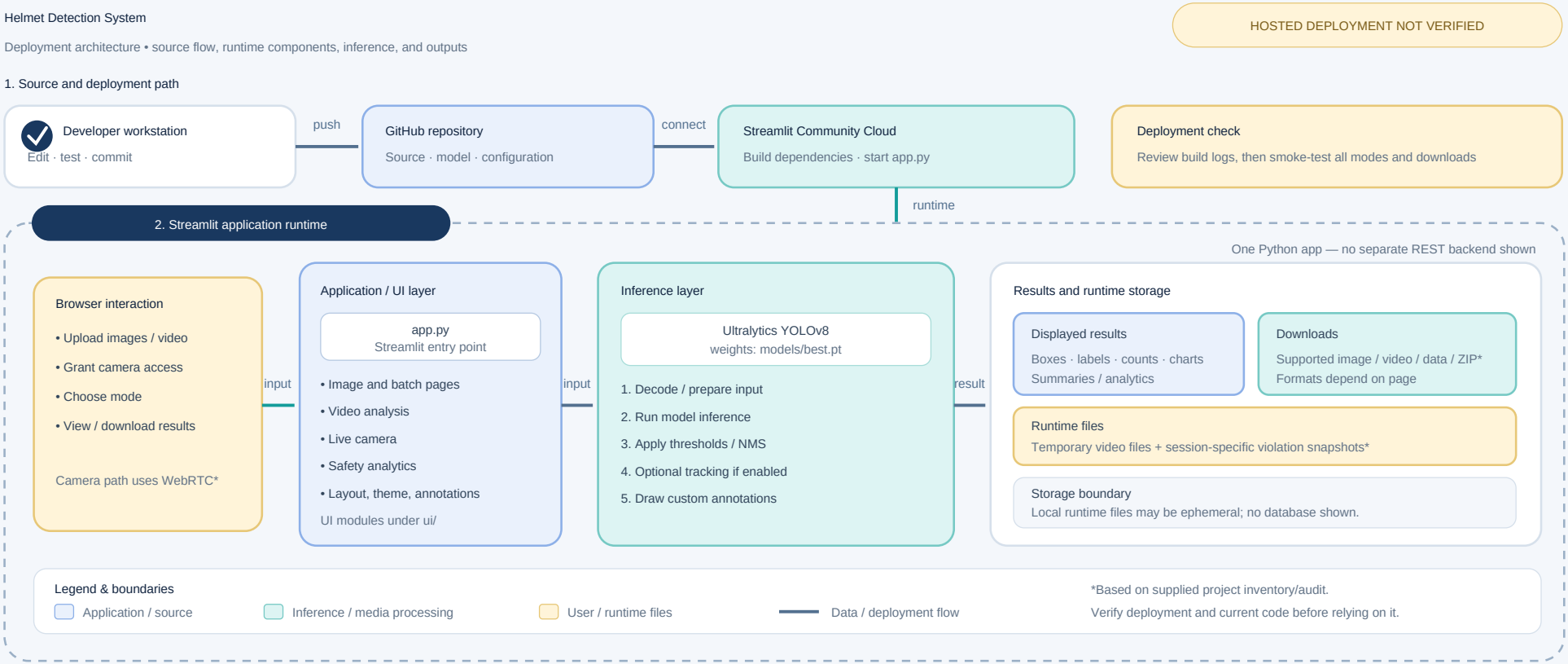
Streamlit entry point plus modular UI pages, layout, theme, styles, and annotations.

Inference layer

Ultralytics YOLOv8 + models/best.pt; filtering/NMS and tracking where enabled.

Output layer

Annotated results, counts, analytics, supported downloads, and runtime files.

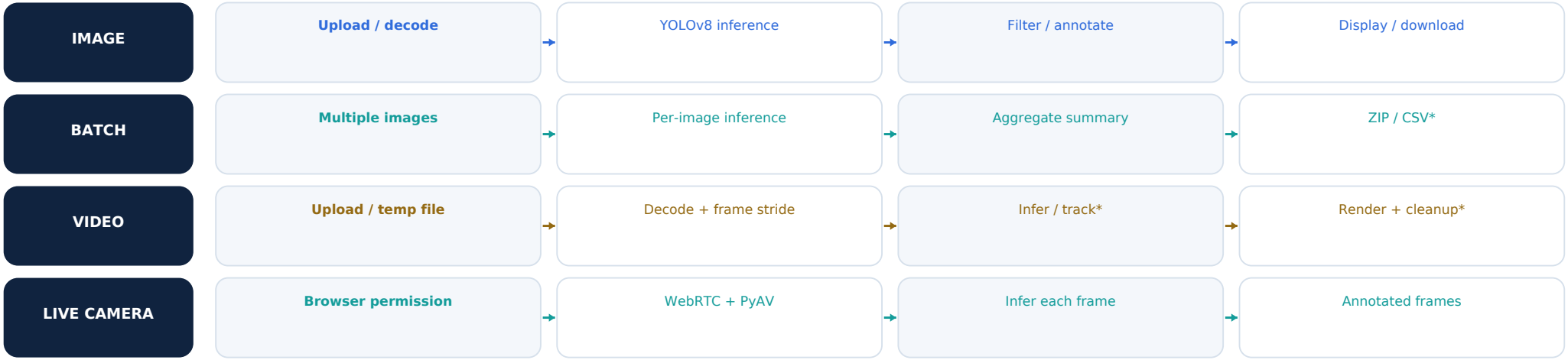


Architecture basis: supplied project inventory and audit • deployment status and live behavior not independently verified.

How data moves through the application

Four input paths converge on model inference; technology choices reflect a Python-centred media-processing workload

PROCESSING PIPELINES



*Details and exports are based on supplied audit/inventory; confirm the deployed commit and smoke-test outcomes.

TECHNOLOGY DECISIONS

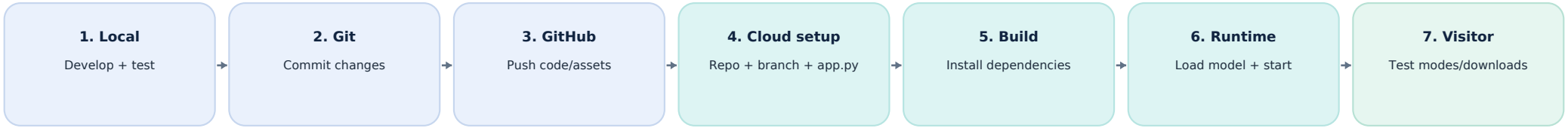
Technology	Project role	Why it fits	Trade-off / limit
Python + Streamlit	App runtime, pages, interaction	One Python-focused app; avoids a separate frontend/backend stack.	Less browser-level control; reruns and server resource use need care.
YOLOv8 + best.pt	Object detection	Runs the trained detector for helmet-related objects/classes.	Quality and speed depend on weights, input size, thresholds and hardware.
OpenCV Headless, Pillow, NumPy	Image/video decode and arrays	Server-side media processing without desktop GUI requirements.	Decoded frames and native libraries still consume RAM and runtime resources.
streamlit-webrtc + PyAV + STUN	Live-camera frames	Provides browser media transport and frame handling.	Permissions/firewall/NAT can block connections; STUN is not a guarantee.
ByteTrack + lap*	Frame-to-frame association	Can maintain object identity over a sequence when enabled.	Adds compute and configuration complexity; verify the active tracker.
Pandas + Altair	Summaries and charts	Integrates tabular aggregation and charts in the app.	Analytics reflect only the available and correctly scoped session data.
Git + GitHub	Version control / source hand-off	Provides repository content for a repository-connected host.	GitHub stores source; it does not run model inference.
requirements.txt + Streamlit Cloud	Runtime dependencies / host target	Reproducible dependency setup and managed Streamlit hosting path.	Python wheels, RAM/CPU limits, cold starts and ephemeral files need validation.

Version note: dependencies are recorded as declared in the supplied requirements listing; verify platform/Python compatibility in the actual build logs.

Deployment, data handling, and interview story

GitHub is the source hand-off; Streamlit Community Cloud is the intended host — validate each live step before claiming production readiness

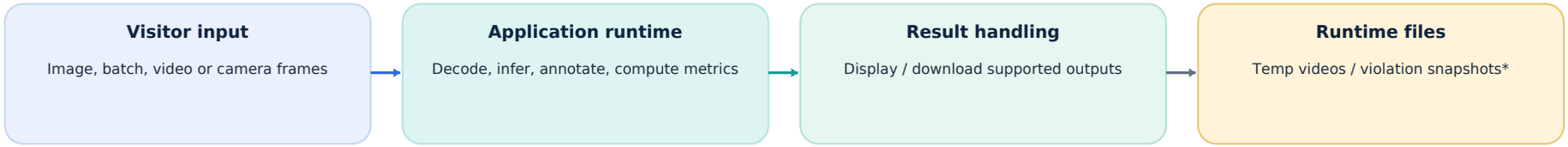
DEPLOYMENT LIFECYCLE / TARGET WORKFLOW



DEPLOYMENT / PUBLIC URL STATUS NOT VERIFIED

The workflow is an intended topology. Confirm repository push, Cloud build, and end-to-end behavior before calling the app deployed.

DATA LIFECYCLE



*Supplied audit reports per-session violation paths and temp-file cleanup. This is not a complete security boundary; hosted disk may be ephemeral.

OPERATIONS & LIMITATIONS

- Runtime sizing: model weights are about 6.23 MB on disk; inference and decoded video frames can use much more RAM.
- Dependencies: check Python compatibility, platform wheels, requirements.txt, and any required system packages in build logs.
- Live camera: test HTTPS, browser permission, and a second network/device. STUN does not guarantee connectivity.
- Storage: verify temp cleanup on success, failure, and abandoned runs; assume hosted local disk can be ephemeral.
- Privacy: keep real secrets out of Git; avoid sensitive uploads until retention and access behavior are reviewed.
- Scope: no database, REST API, Docker or external object storage is shown in this architecture.

STAR METHOD / 60–90 SECOND INTERVIEW STORY

S · SITUATION

Manually reviewing road footage for possible helmet violations can be repetitive, especially across many images and long videos.

T · TASK

Build one practical interface for image, batch, video and live-camera input, with usable detections and summaries.

A · ACTION

Use Streamlit for the Python UI, YOLOv8 for detection, media libraries for decoding, modular pages for workflows, annotations/exports for results, and GitHub for source hand-off to the intended managed host.

R · RESULT

The recorded local pipeline run passed 6 tests. Model and sample media are present in the supplied inventory; hosted behavior and real-world impact still require verification.